

文章笔记

构建 Claude Code 的经验：Skills 的设计与使用

五道口纳什 & Claude

2026 年 3 月 29 日

原文标题：Lessons from Building Claude Code: How We Use Skills

原文作者：Thariq Shihpar (Anthropic Claude Code 团队工程师)

译者：宝玉 @dotey

原文来源：X / Twitter 长文

发布日期：2026 年 3 月 17 日

导读：本文作者 Thariq Shihpar 是 Anthropic 的 Claude Code 团队工程师，也是 Skills 功能的核心推动者之一。Anthropic 内部活跃使用的 Skills 已达数百个，文中的分类体系和编写技巧均来自真实的内部实践。本笔记基于宝玉 (@dotey) 的中文译文整理。

目录

1 Skills 概述	2
1.1 本章小结	2
2 九大 Skills 类别	2
2.1 类别一：库与 API 参考	2
2.2 类别二：产品验证	2
2.3 类别三：数据获取与分析	3
2.4 类别四：业务流程与团队自动化	3
2.5 类别五：代码脚手架与模板	3
2.6 类别六：代码质量与审查	4
2.7 类别七：CI/CD 与部署	4
2.8 类别八：运维手册	4
2.9 类别九：基础设施运维	4
2.10 本章小结	5
3 编写 Skills 的最佳实践	5
3.1 不要说显而易见的事	5
3.2 建一个踩坑点章节	5
3.3 利用文件系统与渐进式披露	5
3.4 不要把 Claude 限制得太死	6
3.5 考虑好初始设置	6
3.6 description 字段是给模型看的	6
3.7 本章小结	6
4 Skills 的高级特性	6
4.1 记忆与数据存储	6
4.2 存储脚本与生成代码	7
4.3 按需钩子 (On Demand Hooks)	7
4.4 本章小结	7
5 Skills 的分享与管理	8
5.1 两种分享方式	8
5.2 管理插件市场	8
5.3 组合 Skills	8
5.4 衡量 Skills 的效果	8
5.5 本章小结	8
6 总结与延伸	8
6.1 核心要点回顾	8
6.2 延伸思考	9
6.3 拓展阅读	9

1 Skills 概述

Skills 已经成为 Claude Code 中使用最广泛的**扩展点** (extension points) 之一。它们灵活、容易制作，分发起来也很简单。但也正因为太灵活，实践中会面临三个核心问题：什么类型的 Skills 值得做？写出好 Skill 的秘诀是什么？什么时候该把它们分享给别人？

Skills 不只是 Markdown 文件

一个常见误解是认为 Skills “只不过是 markdown 文件”。实际上，Skills 是**文件夹**，可以包含脚本、资源文件、数据等内容，智能体可以发现、探索和使用这些内容。在 Claude Code 中，Skills 还拥有丰富的配置选项，包括注册动态钩子 (hooks)。最有意思的 Skills 往往创造性地利用了这些配置选项和文件夹结构。

Anthropic 在梳理了内部所有 Skills 之后发现，它们大致可以归为几个反复出现的类别。最好的 Skills 清晰地落在某一个类别里；让人困惑的 Skills 往往横跨了好几个类别。

1.1 本章小结

Skills 是 Claude Code 的扩展机制，其本质是包含指令、脚本和资源的文件夹，而非简单的文本文件。理解这一点是用好 Skills 的前提。

2 九大 Skills 类别

2.1 类别一：库与 API 参考

帮助正确使用某个库、命令行工具或 SDK 的 Skills。既可以针对内部库，也可以针对 Claude Code 偶尔犯错的常用库。这类 Skills 通常包含：

- 参考代码片段的文件夹
- Claude 在写代码时需要避免的踩坑点 (gotchas) 列表

示例：

- `billing-lib` — 内部计费库的边界情况与容易踩的坑
- `internal-platform-cli` — 内部 CLI 工具的每个子命令及使用场景
- `frontend-design` — 让 Claude 更好地理解设计系统

2.2 类别二：产品验证

描述如何测试或验证代码是否正常工作的 Skills，通常搭配 Playwright、tmux 等外部工具完成验证。

值得投入的验证 Skills

验证类 Skills 对于确保 Claude 输出的正确性非常有用。作者建议**值得安排一个工程师花一周时间专门打磨验证 Skills**。可以考虑让 Claude 录制输出过程的视频，或者在每一步强制执行程序化的状态断言。

示例：

- `signup-flow-driver` — 在无头浏览器中跑完注册、邮件验证、引导流程，每一步都可以插入状态断言的钩子
- `checkout-verifier` — 用 Stripe 测试卡驱动结账 UI，验证发票最终状态
- `tmux-cli-driver` — 针对需要 TTY 的交互式命令行测试

2.3 类别三：数据获取与分析

连接数据和监控体系的 Skills，可能包含带有凭证的数据获取库、特定的仪表盘 ID，以及常用工作流和数据获取方式的说明。

示例：

- `funnel-query` — 注册到激活到付费的转化事件关联，加上存放规范 `user_id` 的表
- `cohort-compare` — 对比两个用户群的留存或转化率，标记统计显著的差异
- `grafana` — 数据源 UID、集群名称、问题到仪表盘对照表

2.4 类别四：业务流程与团队自动化

把重复性工作流自动化为一条命令的 Skills。通常指令比较简单，但可能会依赖其他 Skills 或 MCP (Model Context Protocol, 模型上下文协议)。

日志驱动的一致性

对于这类 Skills，把之前的执行结果保存在日志文件中，有助于模型保持一致性并反思之前的执行情况。

示例：

- `standup-post` — 汇总任务追踪器、GitHub 活动和 Slack 消息，生成只报变化部分的站会汇报
- `create-<ticket-system>-ticket` — 强制执行 schema 加上创建后的工作流
- `weekly-recap` — 已合并 PR + 已关闭工单 + 部署记录，生成格式化周报

2.5 类别五：代码脚手架与模板

为代码库中的特定功能生成框架样板代码 (boilerplate) 的 Skills。当脚手架有自然语言需求、无法纯靠代码覆盖时特别有用。

示例：

- `new-<framework>-workflow` — 搭建新的服务/工作流/处理器
- `new-migration` — 数据库迁移文件模板加常见踩坑点
- `create-app` — 新建内部应用，预配认证、日志和部署配置

2.6 类别六：代码质量与审查

在团队内部执行代码质量标准并辅助代码审查的 Skills。可包含确定性的脚本或工具以保证可靠性，可作为钩子的一部分自动运行或放在 GitHub Action 中执行。

示例：

- `adversarial-review` — 生成一个全新视角的子智能体来挑刺，反复迭代直到发现的问题退化为吹毛求疵
- `code-style` — 强制执行 Claude 默认做不好的代码风格
- `testing-practices` — 关于如何写测试以及测试什么的指导

子智能体 (Subagent)

子智能体是指 Claude Code 在执行任务时启动的另一个独立 Claude 实例。`adversarial-review` 的做法是让一个“没见过这段代码”的新实例来做代码审查，避免原实例的思维惯性。

2.7 类别七：CI/CD 与部署

帮助拉取、推送和部署代码的 Skills，可能引用其他 Skills 来收集数据。

示例：

- `babysit-pr` — 监控 PR，重试不稳定 CI，解决合并冲突，启用自动合并
- `deploy-<service>` — 构建、冒烟测试、渐进式流量切换，指标恶化时自动回滚
- `cherry-pick-prod` — 隔离 worktree，cherry-pick，解决冲突，用模板创建 PR

2.8 类别八：运维手册

接收一个现象 (Slack 消息、告警、错误特征)，引导走完多工具排查流程，最后生成结构化报告。

示例：

- `<service>-debugging` — 把现象对应到工具和查询模式
- `oncall-runner` — 拉取告警，检查常见嫌疑，格式化排查结论
- `log-correlator` — 给定请求 ID，从所有相关系统拉取匹配日志

2.9 类别九：基础设施运维

执行日常维护和运维操作的 Skills，其中一些涉及破坏性操作，需要安全护栏。

破坏性操作需要安全护栏

基础设施运维类 Skills 可能涉及资源清理、依赖变更等破坏性操作。务必在 Skill 中设计确认流程和安全检查，让工程师在执行关键操作时更容易遵循最佳实践。

示例：

- `<resource>-orphans` — 找到孤立 Pod/Volume，发到 Slack，等待确认，级联清理
- `dependency-management` — 组织的依赖审批 workflow
- `cost-investigation` — 存储/出口带宽费用异常调查

2.10 本章小结

九大类别覆盖了从开发到运维的全生命周期：库与 API 参考、产品验证、数据获取与分析、业务流程自动化、代码脚手架、代码质量审查、CI/CD 与部署、运维手册、基础设施运维。好的 Skill 应当清晰地归属于某一个类别，而非试图同时覆盖多个。

3 编写 Skills 的最佳实践

3.1 不要说显而易见的事

Claude Code 对代码库已经非常了解，Claude 本身对编程也很在行。如果 Skill 主要是提供知识，那就把重点放在能**打破 Claude 常规思维模式**的信息上。

frontend-design Skill 的案例

Anthropic 的一位工程师通过与用户反复迭代，构建了 `frontend-design` Skill，专门用来改进 Claude 的设计品味，避免那些典型的套路（如 Inter 字体和紫色渐变）。这是一个很好的“非显而易见知识”Skill 的范例。

3.2 建一个踩坑点章节

踩坑点 (Gotchas) 是 Skill 中信息量最大的部分

任何 Skill 中最有价值的部分就是踩坑点章节。这些章节应该根据 Claude 在使用 Skill 时遇到的常见失败点逐步积累。理想情况下，应持续更新 Skill 来记录新的踩坑点。

3.3 利用文件系统与渐进式披露

Skill 是一个文件夹，应该把整个文件系统当作**上下文工程** (Context Engineering) 和**渐进式披露** (Progressive Disclosure) 的工具。

上下文工程与渐进式披露

上下文工程是指精心设计和管理工作输入给大语言模型的上下文信息，以最大化输出质量。**渐进式披露**借用 UI 设计概念，指不一次性把所有信息塞给模型，而是让它在需要时再去读取，从而节省上下文窗口空间。

具体做法包括：

- 把详细的函数签名和用法示例拆分到 `references/api.md`
- 在 `assets/` 中放模板文件供复制使用
- 提供参考资料、脚本、示例等文件夹

告诉 Claude 你的 Skill 里有哪些文件，它会在合适的时候去读取它们。

3.4 不要把 Claude 限制得太死

过度约束的指令会降低 Skill 的复用性

Claude 通常会努力遵循指令，而 Skills 的复用性很强。不要把指令写得太具体，要给 Claude 它需要的信息，但留给它适应具体情况的灵活性。

3.5 考虑好初始设置

有些 Skills 需要用户提供上下文来完成初始设置。推荐做法：

- 把设置信息存在 Skill 目录下的 `config.json` 文件里
- 如果配置未设置好，智能体会向用户询问
- 如果希望向用户展示结构化的多选题，可以让 Claude 使用 `AskUserQuestion` 工具

3.6 description 字段是给模型看的

description 不是摘要，而是触发条件

当 Claude Code 启动会话时，它会构建所有可用 Skills 及其描述的清单，通过扫描来判断“这个请求有没有对应的 Skill”。因此 `description` 字段描述的是**何时该触发这个 Skill**，而非“这个 Skill 做什么”。好的 `description` 读起来更像 `if-then` 条件，而不是功能说明。

3.7 本章小结

编写 Skills 的核心原则：聚焦非显而易见的知识、持续积累踩坑点、利用文件夹结构做渐进式披露、避免过度约束、设计好初始配置流程、把 `description` 当作触发条件而非功能说明。

4 Skills 的高级特性

4.1 记忆与数据存储

Skills 可以通过在内部存储数据来实现某种形式的记忆。方式从简单到复杂：

- 只追加写入的文本日志文件或 JSON 文件
- SQLite 数据库

例如，`standup-post` Skill 可以保留一份 `standups.log`，记录每一条站会汇报。下次运行时，Claude 会读取历史记录，知道从上次到现在发生了什么变化。

数据存储位置

存在 Skill 目录下的数据可能会在升级 Skill 时被删除。应把数据存在稳定的文件夹中，目前 Claude Code 提供了 `${CLAUDE_PLUGIN_DATA}` 作为每个插件的稳定数据存储目录。

4.2 存储脚本与生成代码

能给 Claude 的最强大工具之一就是代码。给 Claude 提供脚本和库，让它把精力花在组合编排上——决定下一步做什么，而不是重新构造样板代码。

例如，在数据科学 Skill 中放一组从事件源获取数据的函数库，Claude 就可以即时生成脚本来组合这些功能，完成更高级的分析。

```
1 # helpers/data_fetcher.py
2 def get_events(start_date, end_date, event_type):
3     """从事件源获取指定时间范围和类型的事件数据"""
4     ...
5
6 def get_user_cohort(cohort_definition):
7     """根据定义获取用户群"""
8     ...
9
10 def compute_retention(cohort, period="7d"):
11     """计算指定用户群的留存率"""
12     ...
```

Listing 1: Skill 中辅助函数的设计思路（伪代码）

4.3 按需钩子 (On Demand Hooks)

Skills 可以包含只在该 Skill 被调用时才激活的钩子，并在整个会话期间保持生效。这适合那些比较主观、不想一直运行但有时极其有用的功能。

示例：

- /careful — 通过 PreToolUse 匹配器拦截 Bash 中的 `rm -rf`、`DROP TABLE`、`force-push`、`kubectl delete` 等危险操作
- /freeze — 阻止对特定目录之外的任何 Edit/Write 操作，调试时防止意外修改不相关的代码

PreToolUse 钩子

PreToolUse 是 Claude Code 的钩子机制之一，会在 Claude 每次调用工具之前触发。可以在这个钩子里检查即将执行的命令，如果命中危险操作就阻止执行。按需钩子让这种检查只在用户主动调用时才激活，避免在日常开发中造成干扰。

4.4 本章小结

高级特性包括：利用日志或数据库实现跨会话记忆、提供辅助脚本让 Claude 专注于编排逻辑、通过按需钩子在需要时激活安全检查。这些特性让 Skills 从简单的指令文件升级为功能完整的开发工具。

5 Skills 的分享与管理

5.1 两种分享方式

1. **提交到代码仓库**: 把 Skills 放在 `./.claude/skills` 下。适合在较少代码仓库上协作的小团队，但每个 Skill 都会给模型上下文增加一点负担。
2. **插件市场**: 搭建内部 Claude Code Plugin Marketplace，让用户可以上传和安装插件。随着规模扩大，内部插件市场让团队成员自己决定安装哪些 Skills。

5.2 管理插件市场

Anthropic 内部没有专门的中心团队来决定哪些 Skills 进入市场，而是让最有用的 Skills 自然涌现：

1. 上传到 GitHub 的沙盒文件夹，在 Slack 等论坛推荐
2. 获得足够关注后，提交 PR 移到插件市场
3. 正式发布前确保有审核机制

质量控制

创建质量差或重复的 Skills 很容易，在正式发布之前确保有某种审核机制非常重要。

5.3 组合 Skills

Skills 之间可以互相依赖。例如，一个文件上传 Skill 被一个 CSV 生成 Skill 引用。目前这种依赖管理还不被正式支持，但可以直接按名字引用其他 Skills，只要对方已安装，模型就会调用。

5.4 衡量 Skills 的效果

Anthropic 使用 `PreToolUse` 钩子在公司内部记录 Skill 的使用情况，从而发现哪些 Skills 受欢迎，或者哪些触发频率低于预期。

5.5 本章小结

Skills 的分享有仓库级和市场级两种方式。管理的核心思路是让好的 Skills 自然涌现而非自上而下指定。同时要注重质量控制和效果衡量。

6 总结与延伸

6.1 核心要点回顾

本文是 Anthropic Claude Code 团队基于内部数百个 Skills 的实战经验总结，核心要点包括：

1. **Skills 是文件夹，不是文件**: 包含脚本、资源、数据和配置，而非仅仅是一段 Markdown 指令。
2. **九大类别**: 库与 API 参考、产品验证、数据获取与分析、业务流程自动化、代码脚手架、代码质量审查、CI/CD 与部署、运维手册、基础设施运维。好的 Skill 应归属单一类别。

3. **编写原则**：聚焦非显而易见的知识、持续积累踩坑点、利用文件夹做渐进式披露、避免过度约束、把 description 当触发条件。
4. **高级特性**：记忆存储、辅助脚本、按需钩子。
5. **分享与管理**：从仓库提交到插件市场的演进路径，让好的 Skills 自然涌现。

6.2 延伸思考

正如作者所言，Skills 作为 AI 智能体的扩展机制还处于早期阶段。与其把本文当作权威指南，不如视为经过实践验证的技巧合集。大多数好的 Skills 一开始只是几行文字加一个踩坑点，后来因为不断补充 Claude 遇到的新边界情况，才慢慢完善起来。

6.3 拓展阅读

- Claude Code 官方文档中的 Skills 章节
- Skilljar 上关于 Agent Skills 的课程
- Claude Code Plugin Marketplace 文档
- Thariq Shihpar 在 X (@trq212) 上分享的 Claude Code 使用经验